🐣**Practical 1**

Title - Implementing Substitution and Transposition Cipher Using Python

Aim - To design and implement algorithms to encrypt and decrypt messages using classical substitution cipher (Caesar Cipher) and transposition cipher (Columnar Transposition) in Python.

Program 1: Caesar Cipher (Substitution Cipher)

```
def encryption(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            if ch.isupper():
                result += chr((ord(ch) - 65 + shift) % 26 + 65)
            else:
                result += chr((ord(ch) - 97 + shift) % 26 + 97)
        else:
            result += ch
    return result


def decryption(text, shift):
    return encryption(text, -shift)


message = input("Enter Message: ")
shift = int(input("Enter shift value: "))

cipher = encryption(message, shift)
print("Encrypted message:", cipher)

plain = decryption(cipher, shift)
print("Decrypted message:", plain)
```

Program 2: Columnar Transposition Cipher

```
import math

def encrypt(message, key):
    message = message.replace(" ", "")
    rows = math.ceil(len(message) / key)
```

```
    matrix = []
    index = 0

    for i in range(rows):
        row = []
        for j in range(key):
            if index < len(message):
                row.append(message[index])
            else:
                row.append('X')
            index += 1
        matrix.append(row)

    cipher = ""
    for j in range(key):
        for i in range(rows):
            cipher += matrix[i][j]

    return cipher


def decrypt(cipher, key):
    rows = math.ceil(len(cipher) / key)
    matrix = [["" for j in range(key)] for i in range(rows)]

    index = 0
    for j in range(key):
        for i in range(rows):
            matrix[i][j] = cipher[index]
            index += 1

    message = ""
    for i in range(rows):
        for j in range(key):
            message += matrix[i][j]

    return message.rstrip('X')


message = input("Enter message: ")
key = int(input("Enter key (columns): "))

cipher = encrypt(message, key)
print("Encrypted message:", cipher)
```

```
plain = decrypt(cipher, key)
print("Decrypted message:", plain)
```

🐣Practical 2

Title - RSA Encryption and Decryption Using Python

Aim - To implement the RSA algorithm for public key encryption and decryption using Python and understand its properties and security considerations.

```
import math

# Step 1: Select two prime numbers
p = 61
q = 53

# Step 2: Calculate n
n = p * q

# Step 3: Calculate Euler's Totient
phi = (p - 1) * (q - 1)

# Step 4: Select public key e
e = 17

# Check that e and phi are relatively prime
if math.gcd(e, phi) != 1:
    print("Invalid value of e")
    exit()

# Step 5: Calculate private key d
d = pow(e, -1, phi)

# Display keys
print("Public key:", (e, n))
print("Private key:", (d, n))

# Step 6: Take message from user
message = input("Enter a message (single character): ")

if len(message) != 1:
    print("Please enter exactly one character.")
```

```
    exit()

# Convert message into a number
m = ord(message[0])

if m >= n:
    print("Message number must be less than n.")
    exit()

# Step 7: Encryption
c = pow(m, e, n)

print("Original message:", message)
print("Message Number:", m)
print("Encrypted message:", c)

# Step 8: Decryption
decrypted_m = pow(c, d, n)

# Convert number back to character
decrypted_message = chr(decrypted_m)

print("Decrypted message:", decrypted_message)
```

🐣Practical 3

Title - Implementation of Message Authentication Code (MAC) Using Python

Aim - Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

```
import hmac
import hashlib

# Step 1: Enter message and secret key
message = input("Enter the message: ")
key = input("Enter the secret key: ")

# Step 2: Convert into bytes
message = message.encode()
key = key.encode()

# Step 3: Generate MAC
mac = hmac.new(key, message, hashlib.sha256).hexdigest()
```

```
print("\nGenerated MAC:")
print(mac)

# Step 4: Enter received message and MAC
verify_message = input("\nEnter message for verification: ")
verify_mac = input("Enter MAC for verification: ")

# Step 5: Convert received message into bytes
verify_message = verify_message.encode()

# Step 6: Generate MAC for received message
new_mac = hmac.new(
    key, verify_message, hashlib.sha256
).hexdigest()

# Step 7: Compare MACs
if hmac.compare_digest(verify_mac, new_mac):
    print("\nMAC verification successful")
    print("Message is authentic and has not been modified.")
else:
    print("\nMAC verification failed")
    print("Message may have been modified or MAC is incorrect.")
```

🐣Practical 4

Title - Implementation of Diffie-Hellman Key Exchange Algorithm Using Python

Aim - To implement the Diffie-Hellman key exchange algorithm to securely exchange a secret key between two entities over an insecure network.

```
# Public values
p = 23
g = 5

# Alice's private key
a = 6

# Bob's private key
b = 15

# Alice calculates her public key
A = pow(g, a, p)
```

```
# Bob calculates his public key
B = pow(g, b, p)

# Alice calculates the shared secret key
key_alice = pow(B, a, p)

# Bob calculates the shared secret key
key_bob = pow(A, b, p)

# Display results
print("Public prime (p) =", p)
print("Public generator (g) =", g)

print("\nAlice's private key =", a)
print("Alice's public key =", A)

print("\nBob's private key =", b)
print("Bob's public key =", B)

print("\nAlice's shared secret key =", key_alice)
print("Bob's shared secret key =", key_bob)

# Verify the keys
if key_alice == key_bob:
    print("\nKey exchange successful!")
    print("Both parties have the same secret key.")
else:
    print("\nKey exchange failed!")
```

🐣Practical 5

Title - IPSec Simulation Using Python

Aim - To simulate secure IP communication using Python by encrypting a message and verifying its integrity and authenticity.

Terminal - pip install cryptography

```
from cryptography.fernet import Fernet
import hashlib
import hmac
```

```
# Generate encryption key
key = Fernet.generate_key()
cipher = Fernet(key)

# Secret key for HMAC
secret_key = b"mysecretkey"

# Sender's side
message = "This is a secure IPSec communication."
data = message.encode()

# Encrypt the message
encrypted_data = cipher.encrypt(data)

# Generate HMAC
mac = hmac.new(
    secret_key, encrypted_data, hashlib.sha256
).hexdigest()

print("Original message:", message)
print("\nEncrypted message:", encrypted_data.decode())
print("\nHMAC:")
print(mac)

# Receiver's side
received_data = encrypted_data
received_mac = mac

# Generate HMAC for received data
new_mac = hmac.new(
    secret_key, received_data, hashlib.sha256
).hexdigest()

# Verify HMAC before decryption
if hmac.compare_digest(received_mac, new_mac):
    print("\nHMAC verification: Successful")

    # Decrypt the message
    decrypted_data = cipher.decrypt(received_data)
    print("Decrypted message:", decrypted_data.decode())
else:
    print("\nHMAC verification: Failed")
    print("Message may have been modified.")
```

🐣Practical 6
Aim - Digital Signature Using RSA

Title - To implement an RSA-based signature algorithm in Python and verify the integrity and authenticity of a digitally signed message.

Terminal - pip install cryptography

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature

# Generate RSA private key
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Generate public key
public_key = private_key.public_key()

# Create message
message = "This is a secure message."
data = message.encode()

# Create digital signature
signature = private_key.sign(
    data,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("Original message:")
print(message)

print("\nDigital Signature:")
print(signature.hex())

# Verify signature
```

```
try:
    public_key.verify(
        signature,
        data,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("\nSignature Verification: SUCCESS")
    print("The message is authentic and unchanged.")

except InvalidSignature:
    print("\nSignature Verification: FAILED")
    print("The message has been modified.")
```